

by Bob Boonstra, Westford, MA

Dots

You have probably played the game Dots before, although perhaps not in quite some time. My memory of the game dates back to childhood, when we played it in the car while on some long trip. Greg Sadetsky suggested that Dots might make an interesting Challenge, and he wins two Challenge points for that suggestion.

The game starts with a piece of paper with a rectangular grid of NxN dots. The game proceeds with two players alternating turns connecting adjacent dots with a line. Two dots are adjacent if they are in the same row and in adjacent columns, or in the same column and in adjacent rows. The object of the game is to take possession of more squares than does your opponent. Each time a player connects dots that form the fourth edge of a square, the player takes possession of that square. A line can complete zero, one, or two squares. When a player completes one or more squares, s/he is entitled to make another move, and to continue making additional moves as long as a square is completed with each move. The game continues until every square has been formed.

The prototype for the code you should write is:

```
typedef struct Dot {
    short row;      /* row number of dot, 0..boardSize-1 */
    short col;      /* column number of dot, 0..boardSize-1 */
} Dot;

typedef struct DotLine {
    Dot dot1;       /* first dot of a line */
    Dot dot2;       /* second dot of a line */
    /* legal lines are formed by dots in the same row, in adjacent columns, or in the same column
    in adjacent rows */
} DotLine;

void InitDots(
    short boardSize,      /* number of dots per row/col in board */
    Boolean playFirst,    /* true if you play first, false of opponent plays first */
    WindowPtr dotWindow  /* color window where you should draw game results */
);

void OpponentMove(
    const DotLine opponentLine /* line formed by your opponent on previous move */
);

short /* number of lines generated */ PlayDots(
    DotLine yourLines[] /* return the lines you form here */
);

void TermDots(void); /* return any storage you allocated */
```

Play begins with a call to your InitDots routine, where you are given the size of the game board (boardSize), an indicator of who plays first (playFirst), and a pointer to a CWindow (passed as a WindowPtr because that's what most toolbox routines expect). In that window, you will be required to display the progress of the game as it proceeds.

When it is your turn to move, your PlayDots routine will be called. Your code should select the most advantageous move and return it in yourLines[0]. If that move forms a square, you can select an additional move, store it in yourLines[1], and continue as long as squares are formed. PlayDots should return the number of moves you made during your turn.

After your opponent has played, your OpponentMove routine will be called one or more times, once for each move made by your opponent. The move will be provided in the opponentLine parameter, for use in display and in updating your data structures.

After each of your moves, and after notification of each opponent move, you should display the move and the updated game state in the dotWindow. The window should also display the number of squares completed by each player. The details of the display are left to you, as long as the display is correct.

When all of the squares have been formed, your TermDots routine will be called. You should deallocate any dynamically allocated memory and perform any other cleanup required.

The winner will be determined by a round-robin or other fair tournament played with multiple board sizes. Scoring is based on minimizing your point total, calculated as the number of squares that your opponent occupies, plus a penalty of 1% for each millisecond your solution takes to execute.

The Challenge prize will be divided between the overall winner and the best scoring entry from a contestant that has not won the Challenge recently.

This will be a native PowerPC Challenge, using the CodeWarrior Pro 6 environment. Solutions may be coded in C, C++, or Pascal.

Three Months Ago Winner

Congratulations to **Claes Wihlborg** for winning the March DragSort Challenge. The object of this Challenge was to sort a list by dragging items from one position to another, minimizing the cumulative distance that the drag cursor traveled while moving to pick up an item and while dragging it to its new position. Four of the five entries I received sorted my test cases correctly, and three of the four used the same basic algorithm. Claes' entry, however, did so much more efficiently than the others, earning him the Challenge win.

The winning solution moves down the list and picks up the first out-of-order item, dragging it in the same direction and dropping it before the next in-order item. It then continues scanning the list in the same direction, picking up the next out-of-order item and dropping it where it belongs. When it reaches the end of the list, it reverses direction and does the same thing moving backward. It continues to move up and down the list until all of the items are in the correct order. A straightforward and efficient solution.

I evaluated the entries using six test cases using lists ranging up to 6500 elements. Since the problem was based on a hypothetical Usenet application, most of the test cases were based on lists that were ordered correctly except for some relatively small percentage of the list items.

As the best-placing entry from someone who has not won a Challenge in the past two years, **Randy Boring** wins a share of this month's Challenge prize. You don't need to defeat the Challenge points leaders to claim a part of the prize, so enter the Challenge and win Developer Depot credits!

The table below lists, for each of the solutions submitted, the number of penalty points earned by each entry, the total time in milliseconds, and the number of select and drag position moves used to sort the test cases. It also lists the code size, data size, and programming language used for each entry. As usual, the number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges prior to this one.

Name	Points	Time(msecs)	Moves
Claes Wihlborg (29)	29026776	180	29026284
Ernst Munter (721)	29026987	706	29026284
Randy Boring (135)	29027737	1456	29026284
Marco Aiello	84384025	4600	84379428
R. O.	Incorrect	N/A	

Name	Code Size	Data Size	Lang
Claes Wihlborg	704	8	C++
Ernst Munter	2380	80	C++
Randy Boring	2212	145	C++
Marco Aiello	1416	28	C++
R. O.	43432	6120	C++

Top Contestants ...

Listed here are the Top Contestants for the Programmer's Challenge, including everyone who has accumulated 20 or more points during the past two years. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants, the number of wins over the past 24 months, and the total number of career Challenge points.

Rank	Name	Points	Wins(24 mo)	Total Points(24 mo)
1.	Munter, Ernst	294	11	731
2.	Rieken, Willeke	87	3	134
3.	Saxton, Tom	76	2	185
4.	Maurer, Sebastian	68	2	108
5.	Taylor, Jonathan	56	2	56
6.	Shearer, Rob	55	1	62
7.	Wihlborg, Claes	49	2	49

... and the Top Contestants Looking for a Recent Win

In order to give some recognition to other participants in the Challenge, we also list the high scores for contestants who have accumulated points without taking first place in a Challenge during the past two years. Listed here are all of those contestants who have accumulated 6 or more points during the past two years.

Rank	Name	Points(24 mo)	Total Points
8.	Boring, Randy	39	142
9.	Jones, Dennis	12	22
10.	Sadetsky, Gregory	12	14
11.	Downs, Andrew	12	12
12.	Day, Mark	10	30
13.	Duga, Brady	10	10
14.	Fazekas, Miklos	10	10
15.	Flowers, Sue	10	10
16.	Strout, Joe	10	10
17.	Nicolle, Ludovic	7	55
18.	Hala, Ladislav	7	7
19.	Miller, Mike	7	7
20.	Schotsman, Jan	7	7
21.	Widyatama, Yudhi	7	7
22.	Heithcock, JG	6	43